

Agentic Optimization with Coding Agents

Junye Pan, Zian Xiang
junyep2@illinois.edu, xianx2@illinois.edu

Abstract

Frontier language models are increasingly capable of long-horizon software work: they can inspect repositories, edit programs, run commands, interpret failures, and iterate on candidate solutions. Nevertheless, many automated discovery systems still place the large language model inside a relatively fixed outer-loop search scaffold that decides how to sample prior solutions, construct mutations, and retain improvements. This report presents Agentic Optimization, a server-first framework that shifts more of the search behavior to autonomous CLI coding agents while keeping durable state, evaluation, reproducibility, and policy enforcement outside the agent. The architecture combines a thin outer loop, a task API, semantic worker tools, a server-owned control plane, managed runtime environments, artifact-backed candidate snapshots, shared findings, and reusable worker-authored tools. The evaluation interface separates hard-constraint verification, cheap diagnostic probing, and authoritative scoring. In experiments with Codex working with GPT-5.5 as the base coding model, Agentic Optimization reached a state-of-the-art score on the 26-circle packing problem in the unit square, with a best official score of 2.6359836625. We also implemented a Qwen 3.5 4B H200 inference-kernel optimization task; in a separate H200 run, the system produced a preliminary 1.36 $\times$ speedup for an RMSNorm GPU kernel. These results suggest that a minimally scaffolded outer loop can expose useful autonomous optimization behavior while preserving the server-side structure needed for auditability, reproducibility, and controlled evaluation.

Keywords

LLM agents, coding agents, scientific discovery, program evolution, autonomous optimization, evaluation-driven search, agent infrastructure

Contributions. Junye Pan led the project, designed the main framework architecture, implemented most of the system code, and conducted the GPU-kernel optimization task experiment.

Zian Xiang contributed to the system design and conducted the circle-packing experiments.

1 Introduction

Automated scientific and algorithmic discovery has recently moved from offline model prediction toward closed-loop systems in which a model proposes an executable artifact, observes evaluator feedback, and revises the artifact. Early milestones such as AlphaTensor and FunSearch demonstrated that search procedures coupled to strong evaluators can produce nontrivial mathematical and algorithmic discoveries [3, 12]. Newer coding-agent systems broaden this pattern: AlphaEvolve frames discovery as iterative code improvement under evaluator feedback [10]; ShinkaEvolve studies open-ended and sample-efficient program evolution [5]; AVO casts autonomous coding agents as evolutionary variation operators [2];

and Evaluation-driven Scaling argues that feedback loops, rather than one-shot reasoning, are the central scaling axis for scientific discovery [15].

This trajectory raises a systems question. If modern coding agents can inspect code, operate a shell, run tests, analyze traces, and decide which local change to try next, how much of the optimization policy should remain hard-coded in the outer loop? Many systems still confine the model to candidate generation or mutation proposal while an external algorithm chooses parents, context, mutation type, and evaluation schedule. That arrangement is useful when models are weak or when tasks require tight control, but it can suppress agent behaviors that are now becoming valuable: selective retrieval of prior artifacts, opportunistic debugging, self-directed probing, tool creation, local experiment management, and cross-session knowledge reuse.

We propose Agentic Optimization, a framework for giving CLI coding agents more direct responsibility for the inner search process while keeping safety-critical and reproducibility-critical functions server-owned. The core hypothesis is that a minimally scaffolded outer loop can better expose and amplify the optimization capabilities of modern coding agents. In this design, the outer loop defines the task, starts worker sessions, allocates budgets, enforces policy, records events, and owns official evaluation. The worker agent have enough freedom to decide what prior information to inspect, what solution to inherit or modify, when to submit for official scoring, what findings to share, etc.

This report makes three design contributions. First, it articulates a thin-outer-loop architecture for autonomous coding-agent based optimization. Second, it defines a server-owned semantic control plane that represents the artifacts required in an experiments. Third, it separates the score interface into verifier, probe, and submit surfaces, reducing the pressure to expose expensive or private evaluator internals.

2 Related Work

2.1 Evaluation-guided algorithmic and scientific discovery

A central lesson from recent discovery systems is that language-model creativity becomes more reliable when constrained by executable feedback. FunSearch couples an LLM with a systematic evaluator and program archive to search over functions, producing mathematical discoveries that improved known constructions [12]. AlphaEvolve extends this line to a broad evolutionary coding-agent setting, applying direct code edits and evaluators to mathematical, algorithmic, and infrastructure problems [10]. AI-assisted scientific discovery has also been reported in specialized domains, including a neuro-symbolic and tree-search system for a theoretical physics problem [1] and Knuth’s account of AI-assisted reasoning in combinatorial cycle constructions [4]. These works support the premise

that the relevant unit of progress is not a single model response but an evaluated sequence of executable attempts.

2.2 Program evolution and agentic search

Several recent systems explicitly treat LLMs as components in evolutionary or open-ended search. AVO replaces fixed mutation and crossover operators with autonomous coding-agent variation, allowing the agent to inspect lineages, use a knowledge base, and incorporate execution feedback [2]. CORAL moves further toward autonomous multi-agent evolution by combining persistent memory, asynchronous collaboration, evaluator separation, and resource management [11]. Evaluation-driven Scaling studies how parallel exploration, feedback-driven refinement, and local selection can scale scientific discovery loops [15]. Agentic Optimization shares the evaluation-centered orientation of these systems but changes the infrastructure emphasis: the outer loop is deliberately thin, and the durable state is exposed through semantic tools rather than a fixed evolutionary workflow.

2.3 Automated research systems and harness engineering

AutoSOTA frames automated AI research as an end-to-end pipeline that grounds papers to code, repairs environments, runs experiments, generates optimization ideas, and supervises validity [7]. This is close to the target problem class of Agentic Optimization, but AutoSOTA emphasizes an orchestrated multi-agent research pipeline, whereas our design asks how far a CLI coding agent can drive search when backed by a semantic control plane.

A parallel thread studies harnesses: the code and context machinery around models. AutoHarness synthesizes code harnesses that constrain agent actions and reduce invalid behavior [8]. MetaHarness searches over model-harness code using access to prior candidates, scores, and traces [6]. SWE-agent shows that agent-computer interface design materially affects software-engineering agent performance [14]. Voyager and Eureka further demonstrate that executable skill libraries and generated reward code can make feedback-driven LLM systems more adaptive [9, 13]. These works motivate Agentic Optimization’s focus on semantic worker tools, durable shared tools, task knowledge, and explicit evaluation surfaces.

3 Design Goals

Agentic Optimization is organized around six goals.

Agent-owned search behavior. The worker should not merely produce a mutation requested by a controller. It should be able to inspect prior candidates, compare artifacts, run local diagnostics, choose whether to exploit an incumbent or explore a new direction, and decide when a candidate is worth submitting.

Server-owned semantic state. Durable experimental state should not be inferred from directory names or ad hoc log parsing. The server should own semantic records for experiments, assignments, sessions, evaluations, artifacts, jobs, environments, findings, notebook checkpoints, telemetry, shared tools, task knowledge, network policy, leaderboard entries, and incumbents.

Capability-based worker interface. Worker tools should expose capabilities rather than prescribe a fixed algorithm. A worker

can use context, artifact, evaluation, finding, notebook, job, telemetry, etc in any order.

Evaluation separation. The system should distinguish hard-constraint checks, cheap diagnostic probes, and authoritative scoring. This lets workers obtain useful feedback without exposing hidden evaluator internals or spending official benchmark budget on every local attempt.

Reproducible runtime control. Tasks should declare runtime requirements. The server should resolve those requirements into immutable base environments, while worker-requested dependencies should pass through overlay policy rather than silently mutating the official evaluation environment.

4 Framework Overview

The framework is server-first. A worker session is disposable; continuity is provided by server-visible resources rather than an unbounded same-thread conversation. A typical run creates an experiment, generates assignments, starts worker sessions, prepares task environments, materializes a semantic workspace, and allows agents to use the tool surface for context inspection, code editing, etc. Completed valid official evaluations update the leaderboard and derived incumbent, giving later workers a durable checkout target.

This loop is deliberately capability-based. The controller does not encode a fixed numbered workflow such as “retrieve incumbent, mutate, evaluate, reflect.” Instead, it provides a task, a budget, an environment, a policy envelope, and semantic access to accumulated state. The agent’s freedom is bounded by server APIs and environment policy, not the workflow of how to perform optimization.

5 Task API and Evaluation Safeguards

A task package exposes a public contract and three evaluation surfaces. Table 2 summarizes their intended roles.

This decomposition is important because optimization failures are often structured near misses. A binary verifier can reject malformed candidates, while a probe can expose repair-oriented public signals. The official evaluator remains authoritative and can be more expensive, stricter, or more isolated. For path-based official submissions, the control plane snapshots the candidate into an artifact before scoring. This freezes the evaluated object and gives the leaderboard a durable artifact to reference.

Reward hacking and benchmark contamination are treated as system-boundary problems rather than prompt-instruction problems. Hidden tests, private benchmark assets, and scoring code should remain outside the worker-readable workspace. The agent submits candidates through the evaluation API and observes only public-safe feedback. Official scoring defaults to the task base environment; worker overlays may be allowed for verification and probing, but overlay-backed official submission is a separate policy extension.

Table 1: Core framework components.

Component	Responsibility
Coding agent and adapter	Starts CLI coding-agent sessions, prepares the semantic workspace, passes task and policy context, and captures raw execution traces.
Task API	Defines public instructions, candidate contract, runtime specification, verifier, diagnostic probes, and authoritative evaluator.
Control plane	Owens semantic state for experiments, assignments, sessions, environments, jobs, artifacts, evaluations, leaderboards, incumbents, findings, notebooks, shared tools, knowledge, events, telemetry, and network policy.
Semantic worker tools	Provide worker-facing commands such as ctx, artifact, eval, finding, notebook, job, env, telemetry, tool, knowledge, and network.
Environment providers	Resolve task and worker-overlay environments. The current local provider is local_venv; the target reproducibility provider is docker_image.
Artifact and object storage	Stores durable blobs such as candidates, job logs, traces, shared tools, and export bundles while the database records their semantic meaning.

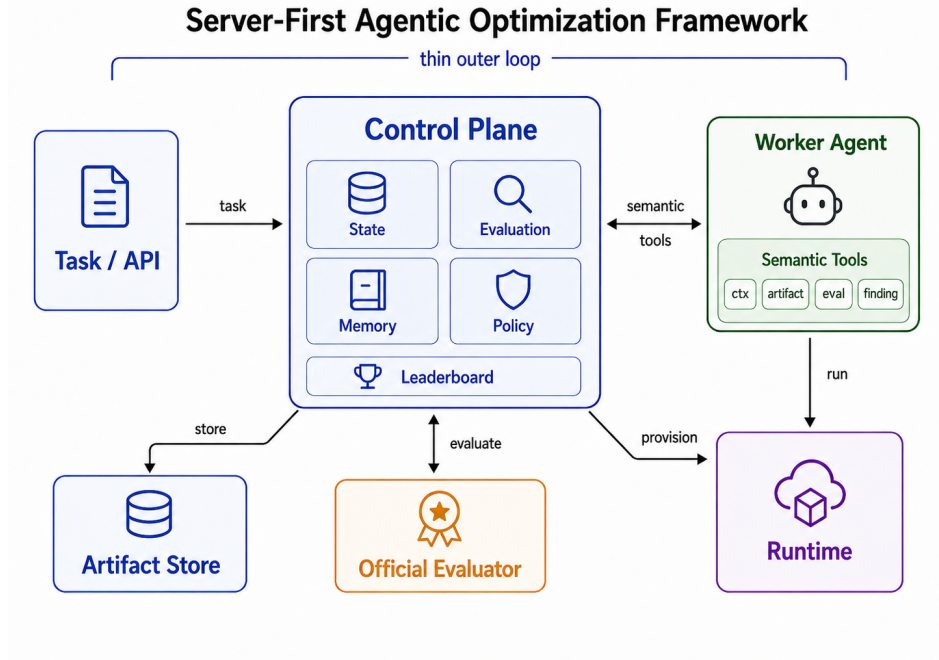


Figure 1: Overview of the proposed server-first agentic optimization framework. A thin outer loop specifies the task, budget, policy, and evaluation boundary, while the server-owned control plane maintains durable state, memory, evaluation records, policies, artifacts, and leaderboard entries. Disposable worker agents interact with the control plane through semantic tools, execute code in managed runtimes, and submit candidates to an isolated official evaluator. This separation preserves agentic search behavior while keeping reproducibility, auditability, and authoritative scoring under server control.

6 Control Plane, Memory, and Artifacts

The control plane has two conceptual layers: a persistence repository and a higher-level service layer. The repository records semantic resources, while the service layer handles operations involving artifacts, jobs, environments, evaluation execution, telemetry, shared tools, task knowledge, and policy checks.

A useful distinction is between *artifact storage* and *semantic state*. Filesystem storage remains appropriate for candidate snapshots, job logs, telemetry logs, workspaces, environment directories, trace bundles, and task packaging. However, those files should not themselves be the experiment’s source of truth. The database records what each file means: whether it is a candidate, shared tool, trace

Table 2: Evaluation surfaces.

Surface	Purpose
verify	Checks hard constraints such as candidate shape, compilation, interface compatibility, and cheap sanity tests.
probe	Provides public diagnostic feedback that is richer than pass/fail verification but not leaderboard-authoritative.
submit	Snapshots the candidate, runs authoritative evaluation, and may publish a leaderboard entry and update the incumbent.

bundle, incumbent checkout, notebook checkpoint, job output, or evaluator snapshot.

Agent-authored memory is split into multiple forms. Local scratch state, such as a `WORKLOG.md`, is useful inside a worker session but is not sufficient for durable sharing. Notebook checkpoints promote useful local notes into server-visible records. Findings are durable agent-authored claims, hypotheses, failures, patterns, or insights. Shared tools are executable helpers that later workers can discover and check out. Task knowledge is separate: curated papers, notes, reference files, or dataset descriptions packaged with the task are read-only inputs, not worker-authored findings.

This design supports long-horizon optimization without relying on a single growing conversation. Later sessions can inspect leaderboards, incumbents, artifacts, findings, notebooks, shared tools, task knowledge, telemetry, and traces through semantic commands. The intended result is a form of cross-session continuity that is auditable and bounded.

7 Semantic Worker Tools

The worker-facing CLI is the principal interface between agent autonomy and server control. The active command surface includes context retrieval, artifact upload and checkout, verification/probing/submission, finding publication, notebook checkpointing, job launch, environment inspection and overlay request, telemetry logging, shared-tool publication and checkout, task-knowledge materialization, and network-policy inspection.

The semantic-tool design has three advantages. First, it reduces archive archaeology. An agent should not need to infer which directory contains the current incumbent or which log file corresponds to an official score. Second, it gives the server a policy gate for operations that affect reproducibility, budget, external network use, or official evaluation. Third, it creates a traceable interaction surface: tool calls can be linked to sessions, evaluations, artifacts, findings, and eventual leaderboard entries.

The accepted next command family is `trace`. Raw coding-agent traces are already useful for debugging and audit, but the target design treats each trace as an immutable, artifact-backed server resource. A trace bundle should include the raw agent event stream, command invocations, command outputs, semantic tool calls, workspace manifest, selected diffs, environment metadata, sandbox metadata, and network-policy status. Trace exports to external observability systems should be mirrors, not the source of truth.

8 Runtime Environments and Network Policy

Runtime control is part of the architecture, not an implementation detail. A task declares a runtime specification, and the server resolves that specification into an environment record. The current base provider is `local_venv`; it prepares a virtual environment, installs declared requirements, verifies required imports, records lock information, and exports runtime variables to worker tool wrappers. Worker-requested dependency changes go through environment overlays, allowing exploration without mutating the immutable task base environment.

The long-term reproducibility target is a `docker_image` provider. A Docker-backed official evaluation should record the candidate artifact, task version, environment image digest, runner metadata, and evaluation request. Tags may be input conveniences, but leaderboard-eligible results should depend on immutable image identities.

Network policy must distinguish semantic control-plane access from external internet access. Semantic tools need access to the control plane even when public web access is denied. Therefore, the policy model has at least two channels: `control_plane_network` and `external_internet`. A local App Server path may require coarse network access to reach the control plane; when it cannot enforce the split, the session should be marked policy-weakened. Docker-backed jobs and future Docker-backed workers should enforce denial of external internet below the agent process, using network isolation or a narrow Unix-socket control-plane relay.

9 Multi-Agent Exploration

Agentic Optimization begins with autonomous worker sessions over server-owned durable state, but it naturally supports controlled multi-agent breadth. Assignments can carry task-defined research directions, islands, or hypotheses. Exploratory workers can receive different directions, while exploiter workers can focus on incumbent improvement or replication. This distinction matters analytically: an exploiter collapsing onto the current frontier is not a failure of diversity, but explorers collapsing into the same method family may indicate premature convergence.

A simple diversity statistic is the entropy of active workers over directions or islands. More detailed analyses can use artifact lineage, finding topics, code-diff clusters, shared-tool use, and evaluation trajectories. CORAL’s emphasis on asynchronous multi-agent execution and shared memory makes this direction especially relevant [11], while Evaluation-driven Scaling suggests that parallel exploration and local selection should be analyzed as first-class scaling dimensions [15].

10 Experiments

We evaluate the current implementation along two axes: a lightweight algorithmic optimization task that can be run end-to-end inside the local control plane, and a heavier GPU-kernel optimization task that exercises the framework’s task-packaging and runtime-contract design. The first task is `circle_packing_26`, where a candidate Python program returns 26 circle centers and radii inside the unit square, and the objective is to maximize the official evaluated sum of radii subject to non-overlap and boundary constraints. The second task is `llm_inference_qwen35_4b_h200`, which packages

an H200-based Qwen 3.5 4B inference optimization problem around replacing selected runtime kernels, beginning with RMSNorm.

For the circle-packing task, we ran Codex workers using GPT-5.5 as the base coding model. Both runs used the server-owned verifier-probe-submit interface, artifact-backed official submissions, local virtual-environment task runtimes, and the semantic worker tools described above. Public internet access was denied by experiment policy. Because the current local App Server backend still requires coarse network access to reach the local control-plane HTTP API, these sessions were recorded as policy-weakened with respect to external-network enforcement; this is a backend limitation rather than the intended long-term policy model.

We compare two search configurations. The linear configuration used one assignment with a total evaluator budget of 250. The multi-agent configuration used five assignments, each with an evaluator budget of 50, corresponding to task-provided research directions: fresh global seed generation, non-row constructive search, topology escape, tolerance-edge polish, and incumbent repair. These directions were used only as assignment context; the controller did not impose a fixed sequence of retrieval, mutation, reflection, or submission steps.

The circle-packing runs show that the current system can support nontrivial autonomous optimization without a hand-coded mutation loop. Workers used the leaderboard, incumbent check-out, findings, artifacts, local helper scripts, probes, and official submissions as needed. In the multi-agent run, different directions produced useful intermediate artifacts and shared tools, while the strongest final result came from tolerance-edge incumbent polishing. The result was not a single one-shot candidate generation event; it emerged from multiple server-visible candidate artifacts, official evaluations, findings, and reusable search tools.

The multi-agent run also illustrates the intended role of durable semantic memory. The control plane recorded 52 artifacts, including 30 candidate artifacts, 21 findings, and 10 shared tools. Later sessions could inspect these records through semantic commands rather than by guessing workspace paths. This supports the design claim that cross-session continuity should be provided by server-owned resources instead of by one long conversation history.

The H200 inference-kernel task is less complete as an experiment but important as a stress test of task design. Unlike circle packing, it requires heavyweight runtime setup, model and tokenizer identity checks, vLLM integration, GPU-specific candidate execution, correctness gates, and performance measurement. The implementation currently packages these concerns into a task boundary with public candidate contracts, verifier and probe logic, and an official evaluation path. Since the full H200 experiment records were produced on another machine and are not synchronized into the local control-plane database, this report treats the task as evidence that the framework can express a heavier benchmark class, not as a finalized empirical comparison.

11 Threats to Validity

Benchmark leakage. Open-ended discovery tasks are vulnerable to public-solution lookup and accidental exposure of hidden evaluator details. The framework addresses this structurally through

hidden evaluators and network policy, but enforcement quality depends on the backend.

Reproducibility gaps. A local virtual environment can record dependencies, but it is weaker than a container image digest for cross-machine reproducibility. Official claims should prefer immutable Docker image identities once the provider is implemented.

Agent variance. Coding agents may differ sharply by model, tool interface, prompt, budget, and sandbox. The framework should record trace, environment, policy, and model metadata so that results can be stratified rather than averaged blindly.

Selection bias. A system designed around tasks with cheap executable feedback may look stronger than it is on tasks where evaluation is slow, noisy, subjective, or difficult to isolate.

Over-instrumentation. Semantic tools can improve auditability, but too many tools or overly verbose context surfaces can become another scaffold that suppresses agent choice. The intended balance is capability-rich but workflow-light.

12 Conclusion

This report presents Agentic Optimization, a server-first framework for autonomous optimization with CLI coding agents. The central design choice is to keep the outer loop thin: it defines tasks, starts sessions, allocates budgets, enforces policy, records state, and owns official evaluation, while the coding agent retains responsibility for the inner search process. Instead of hard-coding a fixed mutation or reflection workflow, the system exposes semantic capabilities for context retrieval, artifact management, verification, probing, official submission, findings, notebooks, jobs, environments, shared tools, task knowledge, and network policy.

Several gaps remain before Agentic Optimization can support stronger reproducibility claims. Even with these limitations, the current implementation suggests a useful direction: preserve agentic search behavior at the worker level, but make the evaluated objects, environments, policies, and memory durable, inspectable, and server-owned.

References

- [1] Michael P. Brenner, Vincent Cohen-Addad, and David Woodruff. 2026. Solving an Open Problem in Theoretical Physics using AI-Assisted Discovery. *arXiv preprint arXiv:2603.04735* (2026). [arXiv:2603.04735](https://arxiv.org/abs/2603.04735)
- [2] Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane, Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger Bringmann, John Tran, Wei Liu, Fung Xie, Michael Lightstone, and Humphrey Shi. 2026. AVO: Agentic Variation Operators for Autonomous Evolutionary Search. *arXiv preprint arXiv:2603.24517* (2026). [arXiv:2603.24517](https://arxiv.org/abs/2603.24517)
- [3] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. 2022. Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning. *Nature* 610, 7930 (2022), 47–53. doi:10.1038/s41586-022-05172-4
- [4] Donald E. Knuth. 2026. Claude’s Cycles. Manuscript. <https://www-cs-faculty.stanford.edu/~knuth/papers/claude-cycles.pdf> Accessed May 12, 2026.
- [5] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution. *arXiv preprint arXiv:2509.19349* (2025). [arXiv:2509.19349](https://arxiv.org/abs/2509.19349)
- [6] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. *arXiv preprint arXiv:2603.28052* (2026). [arXiv:2603.28052](https://arxiv.org/abs/2603.28052)
- [7] Yu Li, Chenyang Shao, Xinyang Liu, Ruotong Zhao, Peijie Liu, Hongyuan Su, Zhibin Chen, Qinglong Yang, Anjie Xu, Yi Fang, Qingbin Zeng, Tianxing Li, Jingbo Xu, Fengli Xu, Yong Li, and Tie-Yan Liu. 2026. AutoSOTA: An End-to-End

Table 3: Comparison on the 26-circle packing problem in the unit square. The objective is to maximize the sum of radii under non-overlap and boundary constraints. Scores marked with “reported” use the precision available in the cited paper.

Method	Search / Agent Setup	Best Score
Prior best-known record before AlphaEvolve	Human / existing computational record, as reported by AlphaEvolve	2.634
AlphaEvolve	Gemini-based coding-agent evolution with evaluator feedback	≥ 2.635
ShinkaEvolve	Open-ended program evolution with weighted parent selection, LLM ensemble, and novelty machinery	2.6359830990
Agentic Optimization linear run	Single Codex worker with GPT-5.5; one assignment; semantic tools; 250 evaluator-run budget	2.6359835698
Agentic Optimization multi-agent run	Five Codex workers with GPT-5.5; one task-defined research direction per assignment; 5×50 evaluator-run budgets	2.6359836625

Automated Research System for State-of-the-Art AI Model Discovery. *arXiv preprint arXiv:2604.05550* (2026). arXiv:2604.05550 <https://arxiv.org/abs/2604.05550>

- [8] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness. *arXiv preprint arXiv:2603.03329* (2026). arXiv:2603.03329 <https://arxiv.org/abs/2603.03329>
- [9] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Eureka: Human-Level Reward Design via Coding Large Language Models. *arXiv preprint arXiv:2310.12931* (2023). arXiv:2310.12931 <https://arxiv.org/abs/2310.12931>
- [10] Alexander Novikov, Ngán Vű, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. *AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery*. Technical Report. Google DeepMind. arXiv:2506.13131 <https://arxiv.org/abs/2506.13131>
- [11] Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu

Liang. 2026. CORAL: Towards Autonomous Multi-Agent Evolution for Open-Ended Discovery. *arXiv preprint arXiv:2604.01658* (2026). arXiv:2604.01658 <https://arxiv.org/abs/2604.01658>

- [12] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625, 7995 (2024), 468–475. doi:10.1038/s41586-023-06924-6
- [13] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023). arXiv:2305.16291 <https://arxiv.org/abs/2305.16291>
- [14] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2405.15793>
- [15] Haotian Ye, Haowei Lin, Jingyi Tang, Yizhen Luo, Caiyin Yang, Chang Su, Rahul Thapa, Rui Yang, Ruihua Liu, Zeyu Li, Chong Gao, Dachao Ding, Guangrong He, Miaolei Zhang, Lina Sun, Wenyang Wang, Yuchen Zhong, Zhuohao Shen, Di He, Jianzhu Ma, Stefano Ermon, Tongyang Li, Xiaowen Chu, James Zou, and Yuzhi Xu. 2026. Evaluation-driven Scaling for Scientific Discovery. *arXiv preprint arXiv:2604.19341* (2026). arXiv:2604.19341 <https://arxiv.org/abs/2604.19341>